

Bug 的秘密生活：揭秘软件存储库中的错误和遗漏

豪尔赫·阿兰达

多伦多大学计算机科学系 10

King's College Road Toronto,

Ontario, M5S 3G4, Canada

jaranda@cs.toronto.edu

吉娜·维诺利亚

Microsoft Research One

Microsoft Way 雷德蒙

德, WA, 98052, 美国

gina.venolia@microsoft.com

抽象

每个虫子背后都有一个故事。发现和解决它的人员需要协调，从文档、工具或其他人那里获取信息，并浏览问责制、所有权和组织结构等问题。本文报告了一项关于围绕 bug 修复的协调活动的实地研究，该研究结合了案例研究研究和对软件专业人员的调查。结果表明，即使是简单的错误的历史也强烈依赖于社会、组织和技术知识，而这些知识不能仅仅通过电子存储库的自动化来提取，而这种自动化提供了不完整且经常错误的协调说明。本文使用丰富的 bug 历史记录和调查结果来识别常见的 bug 修复协调模式，并为工具设计人员和研究人员提供软件开发中协调的启示。

1. 引言

现代大规模软件开发需要每天管理大量错误。修复它们是软件开发人员最常见和最耗时的活动之一 [15]。当 bug 数以千计时，团队成员和研究人员都无法将他们的细节牢记在心。抽象变得必要：项目运行状况由 bug 计数来衡量；生产力取决于一个人关闭的 bug 数量。

在这样的抽象中，很容易忘记每个 bug 背后都有一个故事。发现和解决该问题的人员需要协调，从文档、工具或其他人那里获取信息，并进行导航

通过问责制、所有权和组织结构问题。在流程的每一步都有意识要求、严重的低效率以及提高生产力和质量的机会，但只有当我们超越抽象的数字，进入每个错误丰富、详细的协调历史时，这些才会变得明显。

作为研究人员，我们经常依赖软件项目信息存储库作为主要或唯一证据来源来提取错误和其他工作项的历史记录。它们通常以票据或记录的形式存储在 bug 数据库中。它们提供了方便的工作划分。我们使用项目管理系统的功能，例如审计跟踪和数据字段，以跟踪每个工作项的所有权和上下文。有时，我们使用团队成员之间的电子通信记录以及从人力资源数据库中提取的组织结构数据来丰富工单系统中的历史记录。

然而，到目前为止，使用这些电子存储库作为 bug 或工作项历史的可靠和充分的账户还没有得到适当的验证，我们也没有对 bug 历史背后的协调动态的描述。

本文报告了一项关于围绕 bug 修复的协调活动的实地研究，该研究结合了案例研究研究和对软件专业人员的调查。该研究超越了软件活动的电子存储库，直接与 bug 的关键参与者交谈，以发现通常用于修复 bug 的小组工作模式。它讨论了电子存储库的可靠性，作为软件项目协调研究的基础，并为协调和意识工具的设计提供了一些启示。

· 相关工作

软件专业人员的协调研究面临着需要观察的可能事件和变量太多以及观察和分析资源太有限的问题。选择性的需求渗透到我们的数据收集和分析策略中。为了解决这个问题，作为一个广泛的概括，软件开发中协调的实证研究往往遵循三种方法。

首先，如前所述，有些报告主要基于团队活动的电子痕迹。这些电子痕迹被用作对软件开发工作的观察的替代品。例如，Herbsleb 和 Mockus [10] 使用源代码变更管理系统数据（以及调查）来对分布式软件开发进行建模。Cataldo et al. [4] 使用自动提取的档案数据来计算协调需求。Valetto et al. [19] 挖掘软件存储库以建立组织社会技术一致性的度量。

其次，对一个或几个软件人员的活动进行了详细、丰富的描述。这些通常是人种学观察和飞檐走壁研究的结果，例如 Chong 和 Hurlbutt 关于编程对协调的工作 [5] 以及 Ko 等人 [13] 关于并置团队的信息需求。

第三，有许多软件项目的更广泛、更抽象的报告。这些往往也很详细和细致，但它们侧重于团队或项目层面的趋势和模式。Brooks 对 OS/360 的经典事后分析就是这种情况 [3]，以及 Curtis 等人 [6] 对认知、团队和组织动力学的识别。最近，de Souza 和 Redmiles 使用这种方法来描述软件专业人员的协调性 [7]。

所有这些方法都是必要的，它们为我们提供了有关软件项目中协调的非常有用的信息。但是，据我们所知，我们领域尚未探索过以工作单位为中心的研究协调方法。

在其他学科中，以工作单位为中心的协调研究是有先例的。常见的例子是根本原因分析（RCA）[20] 的研究，它探索了许多过程故障以发现和修复其根本原因。我们的方法在许多方面与 RCA 不同；最重要的是，它对定位流程中的主要故障点不感兴趣（甚至不需要失败），而是对描述和描述流程本身感兴趣。

对软件开发中协调的研究经常将其描绘成一种非正式的、复杂的和依赖于上下文的现象。非正式和无记录的协调活动的重要性已经被多次指出（特别是 Kraut 等 [14]、Perry 等 [17] 以及 Herbsleb 和 Grinter [9]）。文档是正式流程提案的核心，Lethbridge 等人已经研究了文档，他们报告说它经常过时，开发人员很少查阅 [16]。这并不意味着开发人员不需要不断获取项目信息：根据 Singer 等人 [18] 的说法，搜索是开发人员工作的主要类型之一。相反，正如 Hertzum 指出的 [11]，工程师对寻找值得信赖的信息感兴趣，这通常会引导他们找到他们的同事或其他熟悉的来源，而不是文档。

最后，之前关于围绕 bug 仓库协调的研究从不同的角度研究了它们。它们是 Anvik 等人 [1] 关于自动分配 bug 所有权工作的主要数据源。De Souza 等人 [8] 研究了 NASA 的一个项目管理相互依赖关系的方法，并将“问题报告”描述为边界对象，不同的软件专业人员群体将其用于不同的目的。

· 研究设计和执行

我们研究的目标是提供丰富的、上下文化的、以工作项为中心的修复任务中的协调说明。我们有两个主要的研究问题：

首先，软件团队如何协调修复 bug 的过程？bug 的生命周期是怎样的？这项工作中最常见的协调模式是什么？随着时间的推移，他们的解决方案在从事这些工作的团队的社会技术网络图中是如何发挥作用的？

其次，交互的电子痕迹是否提供了足够好的协调画面，或者非持久性知识是否是理解每个错误修复的故事所必需的？

我们分两部分进行了实地研究。第一个是 bug 历史的多案例探索性案例研究 [21]。第二个旨在通过对软件专业人员（开发人员、测试人员和项目经理）的调查来验证我们的案例研究结果。在这两种情况下，我们的数据都来自 Microsoft 产品部门的软件开发。以下部分分别描述了我们研究的两个部分。

3.1 多案例分析

我们案例研究的分析单位是已关闭 bug 的历史。我们将其定义为概念上相关的活动的集合，这些活动在其项目生命周期的某个时刻被总结为错误数据库中的至少一个条目。bug 数据库中的一些记录不是严格意义上的 bug；自从我们的团队以来，我们仍然如此对待他们。某些 bug 具有重复的记录；我们将所有重复项视为同一概念实体的一部分。一些 bug 表现出的症状最初被视为不同的 bug 并单独记录；我们尽可能将它们视为同一缺陷的一部分。

Bug 不一定在 Bug 数据库中创建条目时开始其生命周期，也不一定在标记为“Closed”时结束。其中一些在两个方向上进一步延伸；这种延长的寿命是我们分析单位的一部分。

我们从 Microsoft 的三个主要产品部门中选择了我们的案例。我们的纳入排除标准是：

- 该 bug 被归档在一个 bug 数据库中，并且有关其性质的一些基本信息发布在其数据字段中。

☒ 该错误在我们的观察开始时被标记为“已关闭”。

- 这个错误是新鲜的（它在我们观察开始后的两周内就被关闭了）。

我们的病例是随机选择的。其他数据（例如 bug 的解决机制）不是我们选择标准的一部分，我们没有对它们进行控制。

为了更好地了解用户报告的 Bug 的路径，我们在一个案例中偏离了我们的选择标准：我们联系了客户支持上报工程师，并请求指向来自 Microsoft 上报渠道的 Bug 的指针。

我们所有的案例都遵循相同的方法。首先，我们查询了产品部门的错误数据库，以找到符合我们条件的案例。我们从其电子记录中获得了尽可能多的信息，包括其审计跟踪中的事件、所有 bug 记录的数据字段、其所有者和参与与该 bug 相关的任何作的每个人的数据，以及源代码存储库的链接。

从那时起，我们通过联系最后接触或被 bug 记录引用的人来追溯。如果它们与历史记录无关（由于对 bug 的批量编辑，这是一种常见情况），我们会继续追溯以查找与该 bug 相关的代理。

找到后，我们采访了他们，以了解他们了解该漏洞的历史以及他们从中获得的参与者和文物

信息或他们协调关闭它。当我们被指向一个工件（例如规范文档）时，我们会对其进行分析并追溯到其创建者。当我们被指向其他人时，如果可能的话，我们会联系他们，并与他们重复这个过程。当我们被指向一个持久的通信媒介（例如电子邮件）时，我们获得了一份副本，对其进行了分析，并追溯到其发起者。

在某些情况下，这个过程会走到死胡同，但我们知道还有更多的信息需要找到（例如，因为我们还没有到达最初发现 bug 的地步）。如果是这样的话，我们跳到 bug 记录中的下一个相关参与者，并从该点继续重建 bug 历史。我们总是确保到达故事的开头。

在其他情况下，我们的调查会引导我们找到事件链中如此遥远的人和人工制品，以至于它们与我们的虫子的历史几乎没有关系。在这些情况下，我们做出了主观判断，并停止了对这些分支的探索。

我们的方法在理论上受到了 Hutchins 分布式认知框架对人、人工制品和信息流的关注 [12] 的启发。然而，正如我们所期望的 [2]，我们发现了如此多的信息流实例，以至于在 Hutchins 研究的计算和表征层面执行我们的案例研究是不可行的。我们试图在丰富性和上下文细节与复制和泛化能力之间取得平衡。当我们完全重建了一连串事件，或者当我们部分重建了一连串事件时，我们停止了收集数据，但由于缺乏一些参与者的参与或由于我们的时间限制，进一步进行是不可行的。

我们的数据收集是半结构化的。对于每个案例，我们收集了以下信息：

- 历史记录中的主要参与者和次要参与者及其贡献的列表。

☒ 相关工件和工具的列表。

☒ 错误历史记录中信息流和协调事件的时间顺序列表。

☒ 根据每个案件的特殊性要求提供证据。

☒ 由 bug 数据库中的记录重建的 bug 的历史记录。

- 我们获得的完整电子痕迹集合重建的漏洞历史
- 通过理解所有可用证据（包括我们对参与者的采访）重建的 bug 历史。

表 1 – 案例摘要

Case	Type of Case	How Found	Resolution	Direct Agents	Indirect Agents	Other Listeners	Lifespan (days)	Days w/Events	Events
C1	Documentation	Ad-hoc test	Fixed	6	4	179	320	12	19
C2	Code (security)	Ad-hoc test	Fixed	21	6	3	408	49	138
C3	Build test failure	Automated	Fixed	42	8	291	59	21	141
C4	Code(functionality)	Ad-hoc test	Fixed	6	1	0	7	5	16
C5	Code (install)	User (Beta)	By Design	2	3	0	2	2	12
C6	Code(functionality)	Automated	Fixed	2	4	11	29	6	20
C7	Build test failure	Automated	Fixed	6	7	197	14	6	34
C8	Code(functionality)	Dog food	Won't Fix	2	7	0	2	2	5
C9	Code(functionality)	Automated	Not Repro	5	2	1	2	2	12
C10	Code(functionality)	Escalation	Fixed	23	18	13	35	20	220

我们总共研究了 10 个 bug（包括 Escalation 案例）。我们采访了 26 人。表 1 提供了我们的案例的简要摘要。Direct Agent 是在解决 bug 的过程中执行任务的人员；Indirect Agent 由 Direct Agent 处理，但没有干预。

3.2 调查

我们通过对 Microsoft 软件专业人员的 54 个问题进行调查来验证我们的案例研究结果。我们将其发送给 1,500 名随机选择的 Microsoft 员工，这些员工在开发人员、测试人员和项目经理之间平均分配。我们为参与者提供了赢取 500 美元礼品卡的机会。我们收到了 110 份回复（回复率为 7.3%）；所有回答都是可选的，但对于所有问题，我们至少收到了 100 个回答。我们没有控制受访者的产品划分或人口统计标准。

该调查询问了每个受访者关于他们在解决中发挥主要作用的最近关闭的 bug 的历史。它要求他们检查 bug 数据库中的相应记录，并调出并重新阅读与该 bug 有关的任何电子邮件，以便对 bug 的历史记忆犹新。

该调查分为三个主要部分。在第一个版本中，受访者向我们提供了有关相关 bug 的一般数据。在第二个部分中，我们询问了我们将在第 5 节中介绍的协调模式。在第三个版本中，我们探讨了 bug 数据库中的记录在多大程度上说明了他们的 bug 的全部情况。

一些一般数据问题的结果如图 1-3 所示。虽然不宜使用统计手段将案例研究与调查回复进行比较，但两者大多是一致的，但有一个例外：直接的数量

案例研究与调查中报告的案例研究。调查中直接参与的代理人的四分位数为 3（25%）、4（50%）、6（75%）和 15（100%）。相比之下，我们的 10 个案例中有 3 个案例有超过 15 名直接参与的代理人。我们认为，这种差异表明，我们的调查揭示了比受访者感知的要大得多的错误足迹。

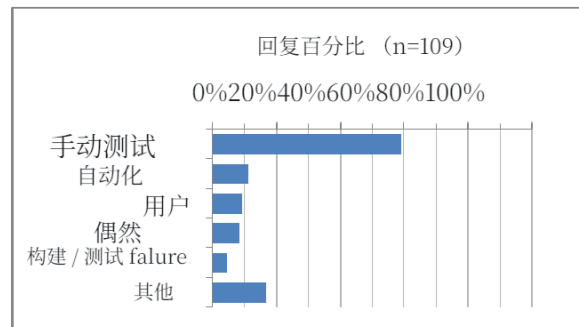


图 1 - 如何发现此错误？

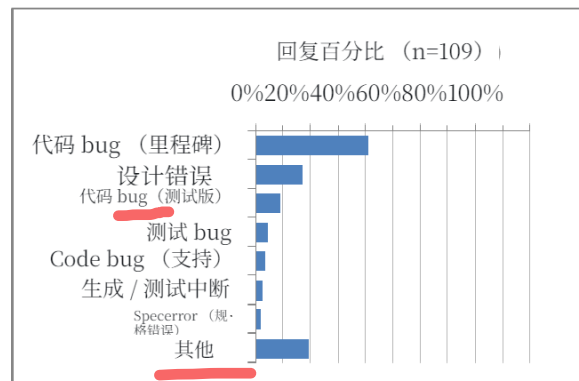


图 2 - 错误类型

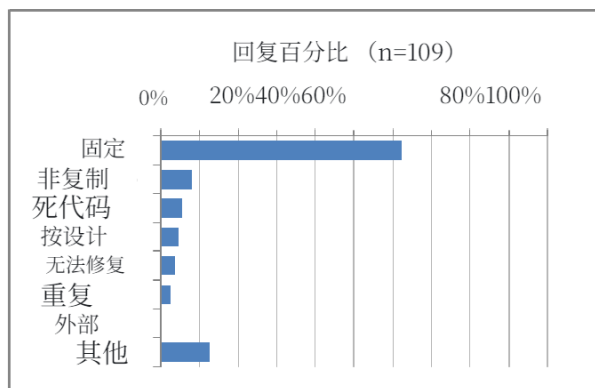


图 3 - 如何解决此错误？

错误和遗漏

从我们的案例中，最引人注目的教训是电子痕迹的严重不可靠性，尤其是错误记录，在我们的 10 个案例中，有 7 个案例是错误或误导性的，而且在每一个案例中都是不完整和不充分的。事实上，即使考虑到我们能找到的所有 bug 的电子痕迹（存储库、电子邮件对话、会议请求、规范、文档修订和组织结构记录），除了一个之外，每个案例的历史记录都遗漏了有关该 bug 的重要细节。

在讨论这些案例中的错误和遗漏之前，我们需要注意，我们认为这个问题不是 Microsoft 特有的粗心或缺乏纪律的结果。我们审查的存储库和文档似乎与同类公司的存储库和文档一样全面，甚至更多。我们将在限制部分更详细地讨论这一点。

4.1 数据收集和分析的级别

我们发现，可以在几个级别上调查 bug 的历史，大致对应于在每个 bug 上需要投入的时间。每个级别都包含了上一层获得的所有信息，以及深入分析的其他发现：

4.1.1 自动分析 bug 记录数据。在第一级，可以使用自动化来获取 bug 历史记录中涉及的代理列表，以及 bug 的生命周期、解决方法、状态更改、如何找到、所有者是谁、哪些代码更改集对应于 bug 以及按时间顺序排列的事件列表等信息。

4.1.2 自动分析电子对话和其他存储库。电子对话的痕迹可以用来构建电子交互的社交网络，并假设它在结构和强度上与参与者的真实通信事件相对应。这些数据可以按参与者、关键字和时间戳进行过滤，以找到（可能）与相关 bug 相关的电子交互。

4.1.3 人类的意义建构。自动化仍然远远低于人类推断数据中的每一个联系和推理证据的能力。

首先，上述电子存储库中通常有大量信息，但这些信息没有正式的联系——发现它需要对证据进行语义上的、非结构化的分析。例如，人员 A 在 bug 记录中的注释可以说明修复“不会解决组 X 的性能问题，这些问题将放在单独的 bug 中”。一个有充分动机的人可以得出结论，A 和 X 的代表之间可能存在讨论，从不同的数据库中提取属于 X 组的人员列表，将其与电子邮件记录匹配以识别相关的对话和涉及的代理，并且，通过试错查询（因为没有提供 bug ID），在数据库中找到后续 bug。

其次，如果我们想理解和改进协调动态，我们需要我们的 bug 历史包括社会、政治和其他隐性信息，这也是软件开发的基本组成部分。这通常是微妙的，并不总是显而易见的，必须从收集到的证据的字里行间来阅读。

4.1.4 参与者对历史的直接叙述。采访 bug 历史的参与者似乎是获取未记录、断开连接或错误信息的最佳途径。访谈可以显著丰富以前级别的数据，只要它们发生在忘记错误的历史之前。它们允许我们验证早期的结论，并检测未存储在记录中但对于了解 bug 历史记录至关重要的事件。

4.2 实践中的级别

我们在列出的四个级别中的最后一个级别进行了案例研究。然而，在整个执行过程中，我们保留了与之前每个数据分析级别相对应的平行错误历史记录，以努力

确定在每个级别获得或更正了多少 Bug 的真实历史记录。

水平之间的差异在数量和质量上都是明显的。下表有助于对它们进行对比。表 2 显示了我们可以在 bug 历史记录中为每个级别记录的事件数。同样，表 3 显示了在每个级别找到的代理（直接和间接）的总数。

表 2 – 事件

Case	Level 1	Level 2	Level 3	Level 4
C1	8	16	17	19
C2	11	11	138	138
C3	19	119	119	141
C4	11	14	15	16
C5	8	11	11	12
C6	12	18	19	20
C7	6	33	34	34
C8	4	4	5	5
C9	7	11	12	12
C10	17	78	149	220

表 3 – 代理

Case	Level 1	Level 2	Level 3	Level 4
C1	7	9	9	10
C2	5	5	27	27
C3	12	38	38	50
C4	5	5	7	7
C5	4	5	2	5
C6	7	7	5	6
C7	7	14	12	13
C8	6	6	15	9
C9	6	7	7	7
C10	8	25	41	41

尽管这些数字支持我们的说法，但它们无法显示 bug 历史记录在不同级别之间的差异程度。不仅仅是更高的级别会产生更长的故事；相反，它们以与协调研究密切相关的方式发生质的变化。以下段落描述了我们观察到的最重要的背离类型。

4.2.1 错误的字段。Bug 记录中的基本数据字段有时不正确。C9 是一个 Test 错误，但它被标记为 Code 错误。C2 和 C4 的某些重复项具有错误的 Status（当其重复项被标记为“已关闭”时，它们仍被标记为“已解决”）。C8 被解析为“Won't Fix”，而它应该被解析为“By Design”。

我们的调查询问了参与者关于他们 bug 的 Resolution 字段以及他们的回答

支持我们的发现。10% 的受访者表示其 bug 的 Resolution 字段不准确。

4.2.2 错误记录中缺少数据。bug 记录中缺少的重要数据包括：指向 C7 和 C10 源代码存储库的链接、指向 bug C2 的重复和相关记录的链接、指向在 C9 中解决原始记录的过程中发现的 bug 的链接、指向规范文档的任何链接（特别是对于情况 C5，由设计解决），重现步骤（C3），为修复错误而采取的纠正措施的声明（C2、C4、C7）和错误根本原因的声明（C7、C9）。

缺少源代码更改集的链接是最有问题的遗漏之一。对于我们 70% 的调查受访者的最后一个错误，修复涉及将代码提交到存储库。但其中 23% 的案例没有从 bug 记录到源的联系。

该调查也支持我们的其他发现。18% 的时间将复制步骤标记为不完整、不准确或缺失。错误根本原因和采取的纠正措施的相应百分比为 26% 和 35%。

4.2.3 人。获取 bug 历史记录中的主要和次要参与者的列表是错误和遗漏的持续来源。对 bug 采取行动的人通常不会在记录或电子邮件通信中提及（C2、C3、C7、C9、C10）。Bug 的自称所有者有时与解决 Bug 无关（C6、C7、C8、C9）。根据电子痕迹，很容易误判参与者的贡献程度：互动的高频率和强度并不意味着高水平的贡献。至少有两次，我们的员工数据库中受访者的地理位置不正确。

在调查中，标记为 Bug 的“所有者”的人仅在 34% 的时间内推动了问题的解决。在 11% 的受访者中，他们与错误无关。

此外，根据我们的回答，在 10% 的时间里，处理 bug 的主要人员不容易通过查看 bug 记录来发现，在 10% 的时间里，他们甚至没有出现在记录中。编辑 bug 字段和历史记录的人员列表在 40% 的时间里只包括一些主要参与者，而 4% 的时间里没有一个。次要参与者的相应数字为 39% 和 38%。在 7% 的情况下，bug 的历史和字段中的所有人都是完全无关紧要的。

4.2.4 事件。期望在错误记录中或通过其

电子痕迹。自然，大多数面对面活动都不会在任何存储库中留下任何痕迹。但在某些情况下，窃听器故事中的关键事件没有留下任何电子痕迹；发现它们的唯一方法是通过参与者的采访。同时，我们所有案例的 bug 记录中记录的一些事件都是噪音或垃圾（例如，批量编辑和后期更正的错误）。事件的时间顺序也是有问题的：Bug 有时甚至在创建记录之前就被解决了（C7），或者在它们被解决很久之后就被关闭了，因为它们被需要关闭它们的人忘记了（C2、C4）。

在调查中，3% 的 bug 在首次提交前至少一个月就已经讨论过，另外 6% 的 bug 在提交前至少一周就已经讨论过。

4.2.5 团体和政治。随着我们从原始数据进一步转向更广泛的协调模式，我们看到团队和部门级别的大多数重要信息只能通过更高级别的分析来找到。在 C4 中，我们发现了一小撮人的文化和习俗与他们所在部门的文化和习俗不同，他们正在被收购后的同化过程中。小组在里程碑和发布方面的状态对发现新 bug 时做出的决策的类型和速度产生了重大影响（例如，对于 C5，团队正在进行“bug 猛烈攻击”，并且每天都有面对面的分类会议，大多数 bug 只得到了一分钟或更短的播出时间）。有时，就像在 C3 和 C7 中一样，Bug 的所有权处于灰色地带，随之而来的是团队间或部门间努力确定所有权和责任。这些问题通常会对 bug 的历史产生相当大的影响，但如果不采访参与者并密切关注电子记录中的细节，我们就不可能了解它们。

4.2.6 理由。如果没有人类的意义建构和参与者的协作，最难回答的问题可能是“为什么”问题：在 C4 中，为什么开发人员选择另一个作为必需的代码审查者，而第三个作为可选的审查者？在 C10 中，为什么在经过几轮每分钟的更新和疯狂的电子邮件之后，错误记录中已经有几周没有活动了？为什么 C2、C4 和 C8 中的 Status 或 Resolution 字段不正确？为什么在 C5 中，分类组得出结论说 bug 不会被修复？为什么测试人员要提交一个 bug C9，即使她怀疑失败很可能是误报呢？我们发现，这些问题的答案，

在访谈中发现的，通常会解开 bug 历史中事件的全部解释。

4.2.7 其他。其他只能在更高层次的分析中找到的事实拒绝分类，但仍然往往是 bug 历史的核心。在 C6 中，测试人员和开发人员在不同的组中独立发现了一个错误；开发人员在不知道该 bug 已被记录的情况下生成了修复程序。在 C4 中，开发人员在发现错误后不久就提交了数百行新代码来修复它；他并没有以极快的速度编写它们，而是从他的旧公司（现已被 Microsoft 收购）的代码中复制它们，并将其“拼接”到相关接口中。在两个案例（C3 和 C7）中，为了解决紧急错误，在一连串的电子邮件中迅速忽略了早期和正确的诊断。一般来说，我们案例池中的 bug 比自动收集和分析它们的电子痕迹所显示的故事要丰富得多，也复杂得多。

· 协调模式

最后，我们的 bug 历史记录丰富、多样且依赖于上下文。它们没有遵循统一的路径或生命周期。这带来了一个问题：我们的第一个研究目标准确地说是描述 bug 的生命周期和修复它们的过程。

我们没有尝试为所有 bug 历史制定一个流程，而是选择描述我们观察到的协调模式菜单。我们选择了那些看起来最频繁的模式，以及那些很少发生但在 bug 的历史中产生重大影响的模式。表 4 列出了它们。

一些模式具有负面影响。例如，我们看到了几个“滚雪球式的线程”和“在公共场合快速发送电子邮件”的案例，这些案例显然效率低下，但它们对我们的参与者来说似乎是例行公事。我们观察到的唯一一次“峰会”对应于一个错误（C3），该错误被负责的发布经理描述为“非常重要”并“威胁要更改我们的发货日期”。我们添加了一种完整性模式（视频会议），尽管我们没有观察到任何实例。另一种模式，“被遗忘”，是由我们的一位调查受访者向我们指出的；它没有包含在我们原来的列表中。

我们询问了调查对象，这些模式是否发生在他们的上一个 bug 中，如果是，它们是否对于解决 bug 至关重要。图 4 提供了他们的回答。最后一列表示 pattern 相对于其频率的感知有用性。

表 4 - 协调模式

Communication media	
Broadcasting emails	Sending a manual or automatic notification to a number of mailing lists to inform their members of an event.
Shotgun emails	Sending an email to a number of mailing lists and individuals in the hope that one of the recipients will have an answer to the current problem
Snowballing threads	Adding people to an ever-increasing list of email recipients.
Probing for expertise	Sending emails to one or few people, not through the “shotgun” method, in the hope that they will either have the expertise to assist with a problem or can redirect to somebody that will.
Probing for ownership	Sending emails to one or few people, not through the “shotgun” method, requesting that they accept ownership of the bug or can redirect to somebody that will.
Infrequent, direct email	Emails sent privately and infrequently among a handful of people.
Rapid-fire email	Bursts of email activity in private among a few people in the process of troubleshooting the issue.
Rapid-fire email in public	Like the above, but with tens or hundreds of people copied as recipients of the email thread, most of them unconnected to the issue.
IM discussion	Using an instant messaging platform to pass along information, troubleshoot, or ping people.
Phone	Phone conversations used to pass along information, troubleshoot, or ping people.
Bug database	
Close-reopen	A bug that is reopened because it had been incorrectly diagnosed or resolved, or because there is disagreement on its resolution or on the team's ability to postpone addressing it.
Follow-up bugs filed	Other bugs were found and filed in the process of fixing this one, or a piece of this bug was filed in a different record as follow-up.
Forgotten	A bug record that goes unnoticed and unattended for long periods.
Working on code	
Code review	The fix for this bug was reviewed and approved by at least one peer.
Two birds with one stone	The fix for this bug also fixed other bugs that had been discovered and filed previously.
While we're there	The fix for this bug also fixed other bugs that had not been discovered previously.
Meeting	
Drop by your office	Getting a piece of information, or bouncing some ideas regarding the issue, face to face informally with a coworker in a nearby office.
Air time in status meeting	The issue was discussed in a regular group status meeting.
Huddle	The issue called for a team meeting exclusively to discuss it.
Summit	The issue called for a meeting among people from different divisions exclusively to discuss it.
Meeting with remote participants	Any meeting where at least one member is attending remotely (could be a huddle or summit meeting).
Video conferences	Any meeting where video was used to communicate with at least one attendee (could be a huddle or summit meeting).
Other patterns	
Ignored fix/diagnosis	A correct diagnosis or fix that was proposed early on and was temporarily ignored by the majority.
Ownership avoidance	Bouncing ownership of the bug or code.
Triaging	Discussing and deciding whether this is an issue worth addressing.
Referring to the spec	At least one concrete and specific reference to a spec, design document, scenario, or vision statement, to provide guidance to solve or settle the issue.
Unexpected contribution	New information or alternatives that come from people out of the group discussing the issue.
Deep collaboration	Two or more people working closely (face to face or electronically) and for a sustained period to unravel the issue.

	Essential (E)	Occurred (O)	(E / O)
Probing for ownership	16%	30%	52%
Summit	1%	2%	50%
Probing for expertise	17%	34%	49%
Code review	30%	66%	46%
Triaging	32%	69%	46%
Rapid-fire email	13%	33%	38%
Infrequent, direct email	18%	49%	37%
Shotgun emails	7%	19%	37%
Drop by your office	23%	64%	36%
IM discussion	11%	32%	35%
Phone	7%	20%	33%
Meetings w/remote part.	4%	12%	33%
Huddle	2%	6%	33%
Air time in status meeting	8%	27%	29%
Close-reopen	4%	13%	29%
Broadcasting emails	10%	35%	27%
Referring to the spec	6%	24%	24%
While we're there	3%	14%	21%
Rapid-fire (in public)	1%	5%	20%
Follow-up bugs filed	3%	18%	16%
Deep collaboration	3%	22%	13%
Snowballing threads	2%	16%	13%
Two birds with one stone	1%	17%	6%
Ownership avoidance	1%	28%	3%
Ignored fix/diagnosis	0%	9%	0%
Video conferences	0%	3%	0%
Unexpected contribution	0%	9%	0%

图 4 - 形态的有用性和频率

我们并不声称我们的模式列表是全面的。但是使用它们来描述错误历史可能会提供有关其协调事件的足够相关信息，同时忽略不相关的细节。此外，正如我们在下一节中讨论的那样，其中一些可以通过软件工具来支持（或者在负模式的情况下，阻止）。

影响

6.1 对于工具设计师

尽管 bug 数据库中常用的状态集（Active、Assigned、Resolved 等）可能有助于管理软件开发，但我们的数据表明，它们对 bug 的真实生命周期的近似值很差。我们认为，对于理解协调和为开发人员设计工具，将 bug 修复活动视为 bug 修复活动更有用。

属于 Bug 生命周期的某个阶段或工作流，但努力满足一个或多个目标。

我们根据案例研究中人们的活动制定了一份目标清单。表 5 描述了它们。并非所有目标都针对每种情况发生，它们也不会严格按顺序发生。

这些目标提供了一个框架来分析协调和项目管理工具和实践的有效性。例如，Assignment of Ownership 经常有问题，尤其是在看似有缺陷的代码没有明确的所有者的情况下。在案例研究中，这种情况在测试脚本中比在功能代码中更常见，而且开发人员没有松散地离开他们的帖子。确保每个工件都有一个活动所有者的工具和实践将减少这个问题。

搜索是另一个有问题的领域。在我们的案例中，它通常会导致“滚雪球式的线程”和“霰弹枪式电子邮件”，它们有时会成功地找到必要的人员或知识，但如果考虑到数百个电子邮件收件人需要花费工时来解析大量消息，而这些消息往往与它们无关，那么效率可能会非常低。

出于协调目的，意识是最需要改进的领域。然而，要弄清楚如何在拥有互连产品的非常大的公司中提供适当水平的认知度并不容易。意识似乎不是在团队层面最需要的，而是在围绕工作项形成社交网络的主要和次要代理之间。

（部分）检测工作网络并允许其成员了解其同事活动的工具应该有助于解决这个问题。

6.2 对于研究人员

我们的案例研究和调查结果指向同一个方向：电子存储库经常保存不完整或不正确的数据。我们基于对 bug 数据库和电子邮件通信的探索得出了这个结论；源代码存储库比这些数据源更容易出错，因为它们对测试人员、项目经理、可用性专家和用户网络视而不见，而这些用户也是协调现象的主要代理。

我们发现的一些具体差异对于大规模的自动协调分析可能是可以接受的。其他问题，例如 4.2.3 中的大多数 People 问题以及从 bug 记录到源代码更改集的缺失链接，要严重得多。

表 5 – 利益相关者在 bug 生命周期内的目标

<i>Discovery</i>	Detecting a difference between reality and expectation. The essential first step to record the unexpected behavior as a bug.
<i>Diagnosis</i>	Understanding the nature, cause, and impact of the bug, as well as the actions that will be taken as a result. We believe it happens in every case, though often tacitly.
<i>Assignment of Ownership</i>	Determining who will be responsible and accountable for the resolution of a bug, both at the group and individual levels.
<i>Search</i>	Finding the appropriate knowledge, resources, and skills. It seems to be often meshed with Assignment of Ownership activities, so that the expert is the owner of the bug –but even the owner may need to reach out for other, more specific bits of expertise and knowledge.
<i>Correction</i>	What we usually think of as “fixing a bug”. Correcting relevant lines of code, changing documentation, scripts, or other artifacts so that reality and expectation match again.
<i>Closure</i>	Determining that the organization is willing to live with the current state of things as related to the bug.
<i>Awareness</i>	Communicating status to relevant participants. It stretches through the whole history of almost every bug.

人们可以采取多种措施来减少电子存储库带来的问题。必不可少的第一步是将它们连接起来。同样重要的是为自动化工具提供一些基本智能，以检测和忽略噪音活动和批量编辑。最后，研究参与组织的标准作可以帮助研究人员确定大多数协调事件是否留下可挖掘的电子痕迹。

但是，即使上述所有问题都得到了解决，仅通过自动化得出关于协调动态的结论也有可能无法感知大规模软件开发的个人、社会和政治因素，从而错过构成软件产品的每个交互的本质。

. 限制

在我们的分析过程中，我们研究了几个在文献中尚未有一致定义的概念。特别是，有人可能会争辩说，我们的协调模式和目标是主观的，界限模糊——例如，我们从未指定过“快速”和“不频繁”电子邮件之间的区别。虽然这是一个有效的批评，但考虑到我们收集的数据，我们的结构是第一次迭代。额外的数据和进一步的迭代应该会改进这些结构，并添加其他有助于更清晰地传达基本概念的结构。

我们的数据完全来自 Microsoft，如果不进行复制，我们的结果对其他公司的有效性就不清楚。众所周知，Microsoft 拥有数万名员工、数百万日常用户和许多互连产品。这些都是塑造协调的力

动力学。但是，Microsoft 对软件存储库和通信媒体的使用似乎与同类公司相似。我们研究中最明显的发现，即 Bug 历史记录的可挖掘版本和真实版本之间的差异，不应该是特定于 Microsoft 的，因为它不取决于企业文化，而是取决于可以经济有效地以电子方式捕获的信息的数量和质量。

对于某些软件开发环境，特别是开源环境，其中所有或大多数信息都以电子方式持久地进行通信，这可能不是什么问题。这项研究的重复将有助于解决这个问题。

. 结论

这项实地研究发现，即使是简单的错误的历史也强烈依赖于社会、组织和技术知识，而这些知识不能仅通过软件存储库的自动分析来提取。自动提取的历史记录提供了事实错误以及不完整和错误的协调说明。本文讨论了缓解此问题的一些策略。

该研究还使用了丰富的 bug 历史记录来发现许多 bug 修复的协调模式，这些模式通过对软件专业人员的调查得到了验证。通过对参与这些模式的人的根本目的的分析，我们得出了 8 个错误修复目标，这些目标可用作设计更好的工具和实践的框架。

尽管我们最初考虑研究功能开发的协调以及错误修复，但一个试点功能案例（本文未报告）被证明属于一个完全不同的领域，并且

我们选择只专注于错误修复。我们计划在未来的工作中探索这两种开发活动之间的差异和互动。

确认

我们感谢 Microsoft Research 的 HIP 团队, 以及 Steve Easterbrook 和 Greg Wilson 在整个研究过程中进行的富有成效的讨论。我们还要感谢参与我们研究的软件专业人士抽出时间。第一作者是 2008 年夏天在 Microsoft Research 的实习生。

引用

[1] J. Anvik, L. Hiew 和 GC Murphy. 谁应该修复此 bug? ICSE'06: 第 28 届软件工程国际会议论文集, 第 361-370 页, 中国上海, 2006 年。

[2] J. Aranda 和 SM Easterbrook. 软件工程研究中的分布式认知: 它能发挥作用吗? 2006 年在加拿大班夫举行的第一届支持大规模软件开发的社会方面 (SSSLSSD) 研讨会上。

[3] F. P. 布鲁克斯。神话般的男人月。艾迪生卫斯理, 1995 年。

[4] M. Cataldo, PA Wagstrom, JD Herbsleb 和 KM Carley. 确定协调要求: 对协作和意识工具设计的影响。CSCW'06: 第 20 届计算机支持合作工作会议论文集, 第 353-362 页, 加拿大班夫, 2006 年。

[5] J. Chong 和 T. Hurlbutt. 结对编程的社会动态。ICSE'07: 第 29 届软件工程国际会议论文集, 第 354-363 页, 美国明尼苏达州明尼阿波利斯, 2007 年。

[6] B. Curtis, H. Krasner 和 N. Iscoe. 对大型系统软件设计过程的实地研究。ACM 通讯, 31 (11): 1268-1287, 1988 年。

[7] C. R. B. de Souza 和 D. F. Redmiles. 软件开发人员对依赖关系和变更的管理的实证研究。ICSE'08: 第 30 届软件工程国际会议论文集, 第 241-250 页, 德国莱比锡, 2008 年。

[8] C. R. B. de Souza, D. F. Redmiles, G. Mark, J. Penix, 和 M. Sierhuis. 协作软件开发中的相互依赖关系管理。在 ISESE 2003: 2003 年国际实证软件工程研讨会论文集, 第 294-303 页, 2003 年。

[9] JD Herbsleb 和 RE Grinter. 拆分组织并整合代码: 重新审视康威定律。在 ICSE'99: 1999 年国际软件工程会议论文集, 第 85-95 页, 美国加利福尼亚州洛杉矶, 1999 年。

[10] JD Herbsleb 和 A. Mockus. 对全球分布式软件开发中的速度和通信的实证研究。IEEE 软件工程汇刊, 29 (6): 481-494, 2003.

[11] M. 赫茨姆。信任对软件工程师评估和信息来源选择的重要性。信息与组织, 12 (1): 1-18, 2002.

[12] E. 哈钦斯。野外认知。麻省理工学院出版社, 1995 年。

[13] AJ Ko, R. DeLine 和 G. Venolia. 并置软件开发团队的信息需求。ICSE'07: 第 29 届软件工程国际会议论文集, 第 344-353 页, 美国明尼苏达州明尼阿波利斯, 2007 年。

[14] RE Kraut 和 LA Streeter. 软件开发协调。ACM 通讯, 38 (3): 69-81, 1995 年。

[15] TD LaToza, G. Venolia 和 R. DeLine. 维护心智模型: 开发人员工作习惯的研究。ICSE'06: 第 28 届软件工程国际会议论文集, 第 492-501 页, 中国上海, 2006 年。

[16] TC Lethbridge, J. Singer 和 A. Forward. 软件工程师如何使用文档: 实践状态。IEEE 软件, 20 (6): 35-39, 2003.

[17] D. Perry, NA Staudenmayer 和 L. G. Votta. 人员、组织和流程改进。IEEE 软件, 11 (4): 36-45, 1994.

[18] J. Singer, T. Lethbridge, N. Vinson 和 N. Anquetil. 对软件工程工作实践的检查。CASCOS'97: 1997 年合作研究中心高级研究中心会议论文集, 加拿大安大略省多伦多, 1997 年。

[19] G. Valetto, M. Helander, K. Ehrlich, S. Chulani, M. Wegman, 和 C. Williams. 使用软件存储库调查开发项目中的社会技术一致性。在 MSR'07: 第四届采矿软件存储库国际研讨会中, 美国明尼苏达州明尼阿波利斯, 2007 年。

[20] P. F. Wilson, L. D. Dell, 和 G. F. Anderson. 根本原因分析: 全面质量管理工具。ASQ 质量出版社, 1993 年。

[21] 尹 R. K. 案例研究研究: 设计与方法 (第 3 版)。Sage, 2003 年。